

Are Online Skill and Memory Modules Always Worth Their Tokens?

A Budget-Constrained Study of Web Agents

Sina Hajimiri^{1,2}, Masih Aminbeidokhti^{1,2}, Jose Dolz², Ismail Ben Ayed²,
Issam H. Laradji^{1,3}, Spandana Gella^{1,4,*}, Nicolas Gontier^{1,*}

¹ServiceNow AI Research, ²ÉTS Montreal

³University of British Columbia, ⁴McGill University

*Equal advising

Online web agents often augment a base actor with memory, workflow, or skill modules. These modules can improve performance, but they also consume test-time tokens, a cost rarely reported alongside the actor’s inference cost. We study online augmentation, where this overhead is paid on every task, and re-evaluate its benefits under a fixed total inference budget. We compare AWM, ASI, and ReasoningBank with a token-matched vanilla baseline that uses the same budget for additional actor steps. Across three WebArena domains and three models, Gemini 3 Flash, GPT-5.4-mini, and Qwen 3.6-27B, the vanilla baseline matches or surpasses all three augmentation methods in aggregate success rate while often using fewer total tokens. We observe a similar trend on WorkArena-L1 with Qwen 3.6-27B, indicating that the effect extends to enterprise knowledge-work tasks. Our results suggest that skills and workflow memory can be useful in specific domains, but their apparent gains often vanish against a budget-matched actor. We further show that run-to-run variance materially affects outcomes and should be reported as a core evaluation criterion for online web agents.

🌐 : <https://sinahmr.github.io/budget-constrained-web-agents>

servicenow

1 Introduction

Web agents are evaluated on how often they solve tasks under a test-time budget. A growing line of work augments a base actor with modules that induce reusable workflows, maintain memory, or distill skills from prior trajectories (Wang et al., 2025a,b; Ouyang et al., 2026; Yu et al., 2026). Following Agent Skill Induction (Wang et al., 2025a) and SkillWeaver (Zheng et al., 2025), we use the term *skill* for executable code, typically a Python function, that wraps a reusable sequence of low-level actions.

These auxiliary modules are not free. They consume tokens, add prompts and control logic, and in the online setting, couple performance to the order in which tasks are processed. Their token

cost is rarely reported next to the actor’s, which makes it hard to tell whether observed gains are justified within a fixed budget. This creates a simple allocation problem: tokens spent inducing, retrieving, verifying, or injecting reusable knowledge are tokens unavailable for direct task execution, such as observing the current page, reasoning over its state, or taking additional actions.

Figure 1 illustrates this tradeoff. Under the same total inference budget, an online augmented agent splits its budget between direct environment interaction and auxiliary memory, workflow, or skill machinery. A budget-matched vanilla actor instead redirects that budget toward additional observe-act steps. The bottom panel previews the main empirical pattern we observe

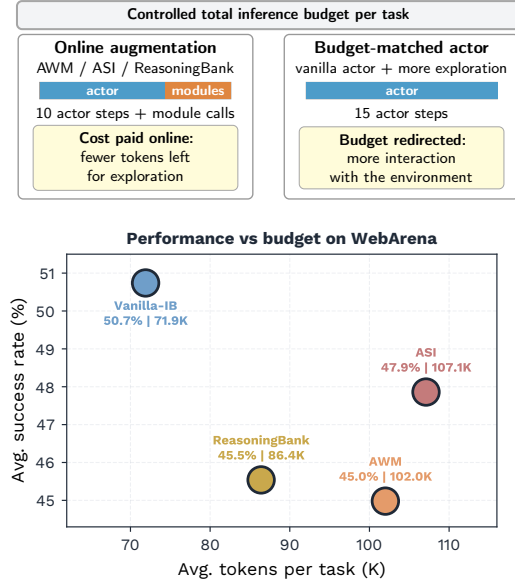


Figure 1 Budget allocation under controlled test-time inference cost. (top) Online augmentation spends budget on workflow, skill, or memory modules, while a budget-matched vanilla actor (Vanilla-IB) spends it on additional observe-act steps. (bottom) On WebArena with Gemini 3 Flash (Google DeepMind, 2025), Vanilla-IB achieves the highest success rate with fewer tokens.

on WebArena (Zhou et al., 2024): once total token usage is counted, a budget-matched vanilla agent achieves the highest average success rate while using fewer tokens than the online augmentation methods.

Prior work on augmentation methods largely assumed that base models lacked the capability to complete tasks end-to-end (Wang et al., 2025b,a); more recent frontier models, however, may no longer face that constraint and may require less external scaffolding. We therefore ask a direct question: under a fixed inference budget, is it better to allocate tokens to online memory and skill modules, or can a vanilla actor achieve comparable performance by spending the same budget on additional interaction steps? We focus on the *online* setting, where agents update their behavior using trajectories collected during evaluation, rather than relying on offline skill libraries constructed before deployment. In this setting, auxiliary costs are incurred repeatedly, the knowledge accumulated by memory or skill

modules depends on the order in which tasks are encountered rather than the full task distribution, and task-level parallelism is constrained by the sequential dependence between tasks.

To study this tradeoff, we compare three online augmentation methods, Agent Workflow Memory (AWM; Wang et al., 2025b), Agent Skill Induction (ASI; Wang et al., 2025a), and ReasoningBank (Ouyang et al., 2026), against a budget-matched vanilla actor. Unlike the augmented methods, which use the default 10-step actor horizon, this baseline allocates the same inference budget to a longer interaction horizon; we refer to it as **Vanilla-Increased-Budget (Vanilla-IB)**. We do not present Vanilla-IB as a new agent scaffold, but rather as a simple budget-aware control designed to test whether the overhead of online augmentation is justified.

We evaluate this comparison on WebArena (Zhou et al., 2024) and WorkArena-L1 (Drouin et al., 2024). WebArena provides the broadest sweep in our study, spanning three models, four agent configurations, and three domains: Shopping, Reddit, and Admin. WorkArena-L1 tests the same budget-matching question in enterprise knowledge-work tasks under Qwen 3.6-27B.

Our main findings are as follows: (1) On WebArena, Vanilla-IB achieves the highest aggregate success rate across all three models we test, while mostly using fewer total tokens than AWM, ASI, and ReasoningBank. (2) On WorkArena-L1, Vanilla-IB remains competitive with the best-performing augmented methods, indicating that the budget-matching question is not specific to WebArena or consumer-style websites. (3) Beyond these aggregate results, we show that single-run success rate hides high task-level variance, and that reporting a method’s performance from a single run is a weak basis for comparison.

These findings suggest three evaluation principles for online web agents. First, comparisons should report *total token usage* across all modules, not only the actor. Second, online augmentation methods should be compared against *baselines within the same budget*. Third, evaluations should report *multi-run variance*, since

agent evaluation introduces stochasticity that aggregate single-run metrics do not capture.

2 Related Work

Web-agent benchmarks. Mind2Web (Deng et al., 2023) and WebArena (Zhou et al., 2024) introduced large-scale benchmarks for realistic multi-step browser interaction. Subsequent work has broadened this benchmark landscape to cover visual grounding in VisualWebArena (Koh et al., 2024), enterprise workflows in WorkArena (Drouin et al., 2024) and WorkArena++ (Boisvert et al., 2024), live websites in Online-Mind2Web (Xue et al., 2025), and continuously generated open-ended tasks in WebArena-Infinity (Zhou, 2026). In this work, we focus on WebArena and WorkArena because they provide interactive environments in which agents must act, observe the consequences of their actions, and complete diverse tasks end-to-end. This makes them well suited for evaluating test-time compute mechanisms, where the cost of additional reasoning steps, tool calls, or memory access must be measured as part of the agent’s deployment-time behavior.

Online workflow, memory, and skill induction.

A growing line of work augments base actors with modules that extract reusable knowledge from evaluation-time trajectories. AWM (Wang et al., 2025b) induces natural-language workflows from successful trajectories and retrieves them for later tasks. ASI (Wang et al., 2025a) synthesizes reusable Python skill functions called as higher-level operations for subsequent tasks. ReasoningBank (Ouyang et al., 2026) distills reasoning strategies from both successes and failures into a growing memory bank queried at test time. PolySkill (Yu et al., 2026) separates a skill’s abstract goal from its concrete realization for more general reuse, and PlugMem (Yang et al., 2026) proposes a task-agnostic memory plugin attachable to any LLM agent. ACE (Zhang et al., 2026) treats the agent’s context itself as an evolving playbook that accumulates and refines strategies through a generate-reflect-curate loop. ERL (Allard et al., 2026) reflects on task trajectories

to extract transferable heuristics, which are retrieved at test time to guide execution on subsequent tasks. All these methods share a common premise: part of the test-time budget is allocated to producing and retrieving reusable knowledge, a cost that is rarely reported alongside the actor’s. Our work asks whether that allocation is always the right choice.

Offline reusable knowledge. A complementary line of work builds reusable knowledge before deployment begins, through an exploration or synthesis phase that is separate from the evaluation run. SkillWeaver (Zheng et al., 2025) discovers and synthesizes Python API skills in an offline phase before deployment. WALT (Prabhu et al., 2026) learns stable tool abstractions reverse-engineered from website structure prior to evaluation, and WebXSkill (Wang et al., 2026) mines parameterized action programs from synthetic agent trajectories. JEF-Hinter (Nekoei et al., 2025) distills offline trajectories into compact, context-aware hints, including from both successful and failed traces, and retrieves relevant hints for the agent’s current state at inference time. Offline approaches amortize discovery or distillation cost over many subsequent uses, which is useful when an agent is deployed repeatedly on the same stable environment. This tradeoff breaks down when the environment is visited infrequently, when its structure changes between discovery and deployment, or when the agent encounters a new domain. These limitations motivate the online setting that is the focus of our work.

LLM capability and the value of scaffolding.

Frontier language models have rapidly improved in capabilities central to web agents: multi-step reasoning, instruction following, and adaptive decision-making under uncertainty. Recent models such as GPT-5.4 (OpenAI, 2026a), Claude Sonnet 4.6 (Anthropic, 2026), and Gemini 3 (Google DeepMind, 2025) can solve complex long-horizon tasks with relatively lightweight scaffolding, often matching or outperforming more heavily engineered agent architectures (Bechard et al., 2026). This rapid progress

has consequences for augmentation methods: gains that were genuine and substantial over weaker models may shrink or disappear as the base actor becomes more capable on its own.

Alternative paradigms and evaluation robustness. Most web agents use a browser, accessibility tree, and fixed action vocabulary. A line of work explores structurally different paradigms: API-based agents substantially outperform browser agents on WebArena, with hybrid agents reaching even higher performance (Song et al., 2025), and terminal-based agents operating through code and programmatic interfaces can match or outperform GUI-based agents on enterprise automation tasks (Bechard et al., 2026). These approaches avoid some browser-skill fragilities, but depend on stable, documented APIs that may not exist. Within the browser-based setting, evaluation reliability is itself a concern: timeouts, page-load failures, and nondeterministic UI behavior can change outcomes independently of agent decisions. Introducing realistic network failures into WebArena, WebVoyager (He et al., 2024), and REAL (Garg et al., 2025) has been shown to cause significant drops in success rates (Kara et al., 2025b). These issues reinforce the importance of multi-run reporting as a first-class evaluation criterion.

3 Experimental Settings

3.1 Tasks and Domains

We evaluate on WebArena (Zhou et al., 2024), a benchmark of realistic multi-step web tasks across isolated website environments. We report results on three domains. **Shopping**, consisting of 187 tasks, is an e-commerce site with product search, comparison, and checkout tasks. **Reddit**, with 106 tasks, is a social discussion platform with navigation, posting, and search tasks. **Admin**, which contains 182 tasks, is the administrator panel of the WebArena shopping site, covering product management, order processing, and account administration. These domains differ in how much their tasks share navigational patterns, which is relevant to whether workflow

or skill reuse can pay off.

The setting is sequential and online: agents attempt tasks in order, and use modules with skill, memory, or workflow components to extract knowledge from earlier trajectories and apply it to later ones.

We also evaluate on the Level-1 collection of WorkArena (Drouin et al., 2024) tasks focused on common enterprise knowledge-work interactions in the ServiceNow platform. This benchmark consists of 33 tasks spanning ServiceNow operations such as dashboards, forms, knowledge-base search, list filtering, list sorting, menu navigation, and service-catalog requests. For WorkArena-L1, we conduct the same online comparison across all task types, using three seeds per task type for each method. This benchmark replaces consumer-style and community websites with enterprise software workflows, while preserving the browser-based interaction setting used by our agents.

3.2 Studied Systems

We compare four agent scaffolds in our experiments. Three methods augment the actor with auxiliary modules while keeping the actor’s interaction horizon to at most 10 steps. The fourth, Vanilla-IB, is a vanilla actor without auxiliary modules but with an extended interaction budget.

AWM (Wang et al., 2025b): online workflow memory, in which an LLM-based auxiliary module induces and retrieves workflows from prior trajectories alongside the actor.

ASI (Wang et al., 2025a): online programmatic skill induction, in which an LLM-based auxiliary module synthesizes, verifies, and reuses Python skill functions.

ReasoningBank (Ouyang et al., 2026): online reasoning memory, in which an LLM-based auxiliary module accumulates and retrieves reasoning strategies from prior successes and failures.

Vanilla-IB: a vanilla actor with its interaction horizon extended to 15 steps and with rule-based pruning of the accessibility tree. The pruning

removes text duplicated between a parent node and its direct children, a common pattern in accessibility trees that inflates context length without adding information. This pruning involves no LLM calls (see Appendix B). We do not present these modifications as a contribution; they are included only to give the vanilla actor more interaction steps under a comparable overall budget.

The 15-step horizon for Vanilla-IB typically does not exceed the total token budget of the augmented methods, because their auxiliary modules consume a substantial fraction of the overall tokens. Exact budget matching is difficult in exploratory web-agent benchmarks such as WebArena and WorkArena. Token usage varies across tasks because agents choose actions dynamically and task complexity differs, so there is no fixed per-task cost to target. Moreover, for augmented methods, a single fixed budget cannot be cleanly divided between the actor and its auxiliary modules. The augmented methods in our comparison also use different aggregate budgets, so there is no single token value against which to match Vanilla-IB. The most practical control lever is therefore the actor’s maximum step count. Setting this horizon to 15 is an approximation rather than an exact budget match. In many tasks, Vanilla-IB could take additional steps while still remaining within the token budget of one or more augmented baselines. In a small number of tasks, the reverse is true: Vanilla-IB spends slightly more tokens than one or more augmented methods. We acknowledge this as a limitation of step-count-based budget control, although such cases are uncommon in our experiments.

3.3 Models and Budget Accounting

We evaluate under three models, GEMINI 3 FLASH (Google DeepMind, 2025), GPT-5.4-MINI (OpenAI, 2026b), and QWEN 3.6-27B (Qwen Team, 2026), each across three independent runs on each domain. On WorkArena-L1, we report the results under QWEN 3.6-27B. For every configuration, we report task **success rate (SR)** and **total token usage**, which aggre-

gates all LLM calls made by the system: actor and any auxiliary module calls (*i.e.*, workflow induction, skill synthesis, memory construction, retrieval, and verification). Token usage is averaged per task in thousands of tokens (K). For both WebArena and WorkArena-L1, metrics are reported as means and standard deviations over three runs. Prior work typically reports metrics on one run, and often does not report token usage; we treat this accounting as a first-class evaluation criterion.

4 Main Results

Performance and token usage. Table 1 summarizes WebArena success rate and total token usage across domains, methods, and models. Across WebArena domains, Vanilla-IB achieves the best average success rate for all three base models and, for Gemini 3 Flash and Qwen 3.6-27B, is also the most token-efficient configuration. The augmented methods therefore do not simply trade extra tokens for higher accuracy; their workflow, skill, or memory modules often increase cost without producing proportional gains in task success. This complicates the interpretation of scaffolded-agent gains: auxiliary reasoning can make a system appear stronger by giving it more computation, but much of that advantage disappears when compared with a vanilla actor that spends a similar budget on additional environment interaction. The effect is strongest for GPT-5.4-mini, suggesting that scaffold quality is itself model-dependent: weaker models can produce noisy workflows, skills, or reasoning memories that are difficult for the actor to ignore. For stronger models, the augmented methods are less prone to bad insights but still do not overtake Vanilla-IB, consistent with modern actors already internalizing many of the planning and navigation abilities that earlier scaffolds attempted to externalize.

Reddit step-sweep. The closest case for augmentation is Reddit with Gemini 3 Flash, where AWM and Vanilla-IB have identical success rates. We probe this case by extending the Vanilla-IB actor horizon further (Figure 2). As the actor

Method	Shopping		Reddit		Admin		Avg.	
	SR (%) $\uparrow$	Tok (K) $\downarrow$	SR (%) $\uparrow$	Tok (K) $\downarrow$	SR (%) $\uparrow$	Tok (K) $\downarrow$	SR (%) $\uparrow$	Tok (K) $\downarrow$
Gemini 3 Flash								
Vanilla-IB	47.77 $\pm$ 1.9	45.7 $\pm$ 0.8	47.48 $\pm$ 0.5	71.7 $\pm$ 2.2	55.68 $\pm$ 1.6	99.0 $\pm$ 4.0	50.74	71.9
AWM	41.18 $\pm$ 2.1	70.8 $\pm$ 3.0	47.48 $\pm$ 1.4	87.2 $\pm$ 2.0	47.43 $\pm$ 0.6	142.7 $\pm$ 5.4	44.98	102.0
ASI	44.74 $\pm$ 1.2	82.8 $\pm$ 0.3	44.97 $\pm$ 2.0	94.6 $\pm$ 4.8	52.75 $\pm$ 1.1	139.4 $\pm$ 11.2	47.86	107.1
ReasoningBank	45.45 $\pm$ 1.4	54.6 $\pm$ 1.3	39.93 $\pm$ 1.4	76.8 $\pm$ 2.8	48.90 $\pm$ 1.9	124.6 $\pm$ 3.3	45.54	86.4
GPT-5.4-mini								
Vanilla-IB	38.50 $\pm$ 0.9	54.3 $\pm$ 0.8	34.59 $\pm$ 4.8	96.2 $\pm$ 6.6	35.89 $\pm$ 1.3	124.1 $\pm$ 8.0	36.63	90.4
AWM	29.95 $\pm$ 1.9	62.0 $\pm$ 2.0	29.56 $\pm$ 1.1	80.4 $\pm$ 12.6	32.24 $\pm$ 3.2	113.4 $\pm$ 16.8	30.74	85.8
ASI	33.15 $\pm$ 0.9	73.4 $\pm$ 9.0	28.93 $\pm$ 2.2	87.2 $\pm$ 4.9	32.96 $\pm$ 3.8	138.5 $\pm$ 10.9	32.14	101.4
ReasoningBank	26.38 $\pm$ 3.9	57.9 $\pm$ 1.6	22.01 $\pm$ 2.9	69.2 $\pm$ 2.9	34.25 $\pm$ 1.8	123.1 $\pm$ 2.3	28.42	85.4
Qwen 3.6-27B								
Vanilla-IB	45.99 $\pm$ 1.4	58.8 $\pm$ 1.5	44.34 $\pm$ 2.5	89.7 $\pm$ 4.3	50.73 $\pm$ 1.4	130.3 $\pm$ 9.5	47.44	93.1
AWM	41.35 $\pm$ 1.9	71.8 $\pm$ 4.5	43.08 $\pm$ 3.0	103.6 $\pm$ 13.8	46.15 $\pm$ 3.1	178.5 $\pm$ 12.5	43.58	119.8
ASI	43.67 $\pm$ 3.5	78.8 $\pm$ 5.5	43.08 $\pm$ 1.4	120.5 $\pm$ 4.7	49.08 $\pm$ 2.3	182.2 $\pm$ 4.2	45.61	127.7
ReasoningBank	39.75 $\pm$ 0.8	65.4 $\pm$ 0.2	41.19 $\pm$ 1.4	99.1 $\pm$ 5.4	47.62 $\pm$ 1.7	156.9 $\pm$ 8.6	43.09	108.0

Table 1 Per-domain success rate (%) and total token usage per task (K) on WebArena. Domain values are means over three independent runs (mean $\pm$ std); per-run details are in Appendix E. Avg. is the mean over all tasks of Shopping, Reddit, and Admin domains.

Method	SR (%) $\uparrow$	Tok (K) $\downarrow$
Vanilla-IB	55.56 $\pm$ 2.9	109.4 $\pm$ 14.8
AWM	53.53 $\pm$ 3.8	102.8 $\pm$ 2.6
ASI	48.49 $\pm$ 4.3	113.8 $\pm$ 10.2
ReasoningBank	55.56 $\pm$ 2.8	120.3 $\pm$ 5.7

Table 2 Success rate (%) and total token usage per task (K) on WorkArena-L1 for Qwen 3.6-27B. Values are means over three independent seeds per task type.

horizon increases, Vanilla-IB steadily improves, matching AWM from 15 steps onward and surpassing it at larger horizons while still using fewer tokens up until 25 steps. This comparison also clarifies the role of step count: the augmented methods already exceed the 15-step Vanilla-IB token budget at their standard 10-step horizon, so giving them the same longer horizon would increase rather than resolve the cost imbalance. The apparent benefit of workflow memory on Reddit is therefore at least partly explained by budget asymmetry rather than by a robust advantage of the auxiliary module itself.

WorkArena-L1. The enterprise setting is more balanced but supports the same basic conclu-

sion. Table 2 shows that, under Qwen 3.6-27B, Vanilla-IB is effectively tied with ReasoningBank for the best success rate, outperforms AWM and ASI, and uses fewer tokens than ASI and ReasoningBank. Thus, Vanilla-IB remains on the success-cost frontier rather than being dominated by an augmented method.

Where Does the Token Budget Go? To identify the source of the token overhead, we decompose total usage into actor tokens from environment interaction and module tokens from workflow induction, skill synthesis, retrieval, and verification, in Table 3. The overhead has two sources. First, augmented methods pay explicit module costs. Second, and less obvious, augmentation methods also inflate the actor’s prompt by injecting retrieved workflows, memories, or available skill functions at each step. This implicit cost cannot be removed by making module calls cheaper: as long as retrieved knowledge is inserted into the actor context, the per-step actor cost rises with the size of that content. Augmentation therefore imposes a double cost, combining auxiliary inference with larger actor prompts.

Method	Shopping				Reddit				Admin			
	Actor		Modules		Actor		Modules		Actor		Modules	
	Prompt	Compl.	Prompt	Compl.	Prompt	Compl.	Prompt	Compl.	Prompt	Compl.	Prompt	Compl.
Vanilla-IB	45.2	0.5	0	0	71.3	0.4	0	0	98.4	0.6	0	0
AWM	64.1	0.4	6.1	0.2	82.9	0.4	3.7	0.2	135.0	0.5	6.9	0.3
ASI	61.8	0.5	20.0	0.5	84.6	0.4	9.3	0.3	118.1	0.5	20.2	0.6
ReasoningBank	36.6	1.1	16.5	0.4	53.3	1.0	22.1	0.4	96.9	1.2	26.1	0.4

Table 3 Token breakdown by component under Gemini 3 Flash on WebArena, per task in thousands (K). Actor: tokens from LLM calls made during interaction steps. Modules: tokens from auxiliary components (workflow induction, skill synthesis, retrieval, verification). Both are further split into prompt and completion tokens. Values are means over three runs. Vanilla-IB has no module calls by construction.

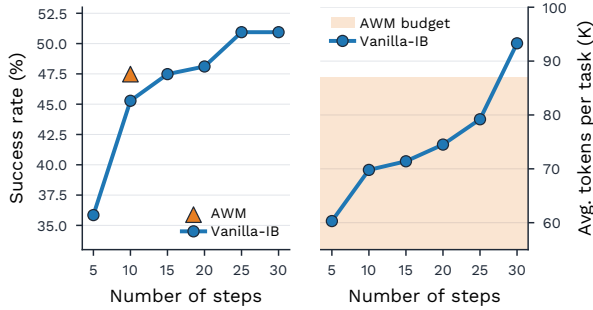


Figure 2 Vanilla-IB horizon sweep on Reddit under Gemini 3 Flash. Success rate (left) and total tokens per task (right) are reported as a function of interaction steps. The available budget is the token usage of the best-performing augmentation baseline, in this case AWM, shaded in orange (right). At the 10-step mark, Vanilla-IB SR is below AWM, which in isolation might seem to justify augmentation. However, the right panel shows AWM already spends roughly twice as many tokens at that point. Extending the Vanilla-IB horizon up to 25 steps closes this gap: Vanilla-IB surpasses AWM in SR while still using fewer tokens than AWM.

Headroom for Vanilla-IB. We set the Vanilla-IB horizon to 15 steps as a clean, domain-agnostic choice, but in most cases it is a conservative setting. As the Reddit sweep shows (Figure 2), token usage at 15 steps is still well below AWM’s budget, and success rate continues to rise before saturating around 25 steps. The unused headroom is even larger on Shopping, where the token gap to augmentation methods is wider. Together, these results suggest that comparisons should be made along the axis of total token cost rather than nominal step count. AWM evaluated at 10 steps already spends more tokens than Vanilla-IB at 15 steps, so extending

its actor horizon would only widen this cost gap rather than close it. For any step budget at which augmented methods are evaluated, a budget-matched vanilla baseline can be obtained by increasing the Vanilla-IB horizon accordingly; the same token-efficiency imbalance therefore persists across operating points.

5 Audit Beyond Aggregate Success Rate

5.1 Run-to-Run Variance

Multi-run reporting. A single-run aggregate success rate hides properties that matter for on-line web agents. Even at a fixed temperature, LLM outputs vary across runs due to hardware-level non-determinism in distributed inference. A given task may succeed in one run and fail in another for reasons unrelated to agent capability: a different token sample produces a different action sequence, which may or may not navigate the same interface in the same way. Another source of variance can be the web environment itself: BrowserGym (Le Sellier de Chezelles et al., 2025) and Playwright (Microsoft, 2025) can encounter timeouts, slow page loads, or intermittent server errors, and prior work shows that injected web failures can sharply reduce success rates on web tasks (Kara et al., 2025a,b). As a result, two runs of the same agent on the same task set can produce nearly identical aggregate success rates while disagreeing on a meaningful fraction of individual tasks. This aggregate stability is reassuring for reproducibility but hides large task-level noise that single-run evaluations do not expose.

Multi-run reporting helps absorb both sources of noise and provides a more reliable picture of what the agent can actually do. A single reported success rate is an incomplete statistic: it conflates genuine agent capability with the particular set of environment states encountered in that run, and this conflation is especially dangerous when comparing methods that differ in the number or structure of their LLM calls, since some methods have more opportunities to encounter a bad environment state. We therefore perform our experiments over three runs, and the detailed per-run results are provided in Appendix E.

Any-of-3 and All-of-3. To expose the task-level instability, we compute two complementary bounds: *Any-of-3*, the fraction of tasks that succeed in at least one of the three runs (an OR across runs); and *All-of-3*, the fraction that succeed in all three runs (an AND across runs). Table 4 reports both as illustrative bounds on the latent capability that single-run averages hide, and shows that the gaps are relatively large for all the methods. Single-run success rate is therefore an incomplete estimate of task-level capability, especially for methods whose learned state changes across the evaluation sequence.

5.2 Programmatic Skill Fragility

A structural limit of programmatic skill approaches such as ASI is that a synthesized skill is tied to the accessibility-tree state in which it was created. These skills often wrap a sequence of low-level actions and refer to UI elements by BrowserGym accessibility-tree IDs (BIDs), which are assigned dynamically. BIDs are not stable across interactions. After the first action in a skill, the page may reload or update, and later BIDs may then point to different elements or to nothing at all. The function then either fails silently, throws an error that disrupts the agent’s trajectory, or interacts with the wrong element.

A vanilla agent that queries the LLM at each step adapts to state changes naturally, since it observes the current accessibility tree before choosing each action. A function-based approach lacks this adaptability unless it inserts an extra LLM

call inside the function at each sub-step, in which case its efficiency justification largely disappears. Appendix D provides explanations of this and related failure modes across all three augmented methods, including concrete task-level examples where augmented agents fail and Vanilla-IB succeeds.

5.3 Effect of Accessibility-Tree Pruning

Vanilla-IB combines two changes relative to a plain 10-step agent: an extended 15-step actor horizon and rule-based accessibility-tree pruning (Appendix B). Table 5 isolates their individual contributions.

The extended horizon is the main success-rate improvement, while pruning is primarily a token-reduction mechanism. Pruning consistently reduces token usage at both horizons and more than offsets the cost of five additional interaction steps, so Vanilla-IB uses fewer tokens than the plain 10-step baseline while matching or improving success rate. Its effect on success rate is smaller and domain-dependent, which is expected: the purpose of pruning is to remove redundant accessibility-tree text, not to add new reasoning capability. When pruning helps, we suspect it is because removing duplicate entries makes the context more information-dense and shortens token distances between related elements, such as labels and fields. These effects may make actionable content easier to track, but the mixed domain results suggest that this benefit is not uniform.

6 Practical Advantages of Stateless Evaluation

Beyond success rate and token cost, two practical properties differ between Vanilla-IB and the augmented methods, both stemming from the absence of cross-task state.

Parallelism. Vanilla-IB treats tasks as independent episodes and maintains no shared cross-task state. As a result, with N workers, tasks can be executed concurrently, reducing end-to-end wall-

Method	Gemini 3 Flash				GPT-5.4-mini				Qwen 3.6-27B			
	Any	$\Delta_{\uparrow}$	All	$\Delta_{\downarrow}$	Any	$\Delta_{\uparrow}$	All	$\Delta_{\downarrow}$	Any	$\Delta_{\uparrow}$	All	$\Delta_{\downarrow}$
Vanilla-IB	54.01	+6.2	42.25	-5.5	46.52	+8.0	27.81	-10.7	54.01	+8.0	38.50	-7.5
AWM	47.06	+5.9	35.83	-5.4	37.43	+7.5	21.39	-8.6	49.73	+8.4	33.16	-8.2
ASI	51.34	+6.6	37.43	-7.3	40.64	+7.5	26.74	-6.4	52.41	+8.7	36.36	-7.3
ReasoningBank	53.48	+8.0	38.50	-7.0	35.29	+8.9	17.64	-8.7	47.06	+7.3	34.22	-5.5

Table 4 Any-of-3 and All-of-3 success rates (%) on the Shopping domain of WebArena across three models. Any-of-3 is the fraction of tasks succeeding in at least one run; All-of-3 is the fraction succeeding in all three runs. $\Delta_{\uparrow}$ and $\Delta_{\downarrow}$ are the gaps from the three-run mean (Table 1).

Configuration	Shopping		Reddit		Admin	
	SR (%) $\uparrow$	Tok (K) $\downarrow$	SR (%) $\uparrow$	Tok (K) $\downarrow$	SR (%) $\uparrow$	Tok (K) $\downarrow$
10 steps	45.81 $\pm$ 1.5	48.0 $\pm$ 1.1	45.28 $\pm$ 1.0	75.2 $\pm$ 6.4	48.72 $\pm$ 0.3	103.5 $\pm$ 7.7
10 steps, pruned	46.35 $\pm$ 2.2	41.7 $\pm$ 0.5	47.17 $\pm$ 0.9	70.8 $\pm$ 1.6	51.65 $\pm$ 1.0	85.5 $\pm$ 3.9
15 steps	47.24 $\pm$ 2.8	54.0 $\pm$ 1.1	47.48 $\pm$ 2.2	77.7 $\pm$ 6.7	57.14 $\pm$ 0.0	119.8 $\pm$ 16.1
15 steps, pruned	47.77 $\pm$ 1.9	45.7 $\pm$ 0.8	47.48 $\pm$ 0.5	71.7 $\pm$ 2.2	55.68 $\pm$ 1.6	99.0 $\pm$ 4.0

Table 5 Effect of extending the vanilla actor horizon from 10 to 15 steps and rule-based accessibility-tree pruning, across all three domains of WebArena under Gemini 3 Flash. Values are means over three runs (mean $\pm$ std). The last row corresponds to Vanilla-IB.

clock time by up to a factor of N . In contrast, AWM, ASI, and ReasoningBank are inherently sequential: each task must finish before the next begins because the memory or skill state derived from its trajectory is reused in subsequent tasks. This makes parallel deployability an additional practical advantage of Vanilla-IB, beyond its token efficiency.

Fault tolerance. Web benchmarks are run against live or simulated website environments that are not always perfectly reliable: a server may become temporarily unresponsive, a page may time out, or a browser session may crash mid-task for reasons unrelated to the agent (Kara et al., 2025a). For Vanilla-IB, any such failure is local: the affected task can simply be re-run in isolation and the result substituted, since no other task depends on its outcome. For AWM, ASI, and ReasoningBank, this repair is not possible in general. If task k fails due to an environment glitch, the knowledge state used by all subsequent tasks $k+1, k+2, \dots$ was already formed without a potential successful trajectory from task k . Re-running task k in isolation does not retroactively correct the downstream state;

a proper repair would require re-running the entire experiment, which can be expensive. A single transient environment failure can therefore silently degrade the reported performance of an online augmentation method in a way that is hard to detect. In our experiments, when we identified a failure caused by the evaluation infrastructure rather than the agent’s decisions, such as a network issue or browser crash, we re-ran the affected task immediately so that the augmentation methods could still potentially induce knowledge from it, and so that the reported numbers reflect each method’s performance under its intended operating conditions.

7 Conclusion

We re-evaluated online web-agent augmentation under a fixed total inference budget, asking whether test-time tokens are better spent on memory, workflow, or skill modules, or directly on additional actor steps. Across three WebArena domains and three models (Gemini 3 Flash, GPT-5.4-mini, and Qwen 3.6-27B), a token-matched vanilla actor matches or surpasses AWM, ASI, and ReasoningBank in aggregate

success rate while often using fewer total tokens. We observed the same trend on WorkArena-L1 with Qwen 3.6-27B, indicating that the effect extends beyond consumer web tasks to enterprise knowledge-work settings. These results do not imply that skills or workflow memory are useless: they can provide gains in domains where task structure supports reuse. Rather, our findings show that such gains are conditional and often disappear once augmentation overhead is compared against a budget-matched actor. We also show that run-to-run variance materially affects outcomes, motivating the inclusion of multi-run experiments as a core evaluation criterion rather than an optional diagnostic. Overall, our results argue for stronger evaluation hygiene in online web-agent research: total test-time cost should be reported transparently, augmented agents should be compared against budget-aware vanilla baselines, and robustness analyses should accompany aggregate success rates.

8 Limitations

This paper reports a focused empirical comparison on WebArena under three models (Gemini 3 Flash, GPT-5.4-mini, and Qwen 3.6-27B), plus WorkArena-L1 under Qwen 3.6-27B, and several limitations follow from this scope.

Our WebArena conclusions are restricted to three of WebArena’s five domains (Shopping, Reddit, and Admin), and methods might behave differently on the other domains. Our WorkArena-L1 results are restricted to Qwen 3.6-27B and three seeds per task type. More broadly, the results should not be generalized to all web environments. Settings with more homogeneous task distributions, longer deployment horizons, or environments built around stable high-level APIs may yield a more favorable tradeoff for augmentation methods. Besides, our budget-matching argument applies to online augmentation only; offline methods such as SkillWeaver (Zheng et al., 2025) and WALT (Prabhu et al., 2026) amortize skill-discovery cost over many uses, and a fair comparison there requires different accounting.

References

- Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. Experiential reflective learning for self-improving LLM agents. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems*, 2026. URL <https://openreview.net/forum?id=hQgSl6kj1W>.
- Anthropic. Introducing Claude Sonnet 4.6. <https://www.anthropic.com/news/claude-sonnet-4-6>, February 2026. Model announcement.
- Patrice Bechard, Orlando Marquez Ayala, Emily Chen, Jordan Skelton, Sagar Davasam, Srinivas Sunkara, Vikas Yadav, and Sai Rajeswar. Terminal agents suffice for enterprise automation. *arXiv preprint arXiv:2604.00073*, 2026. doi: 10.48550/arXiv.2604.00073. URL <https://arxiv.org/abs/2604.00073>.
- Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *Advances in Neural Information Processing Systems*, volume 37, pp. 5996–6051, 2024. doi: 10.52202/079017-0195. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/0b82662b6c32e887bb252a74d8cb2d5e-Paper-Datasets_and_Benchmarks_Track.pdf.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, volume 36, pp. 28091–28114, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/5950bf290a1570ea401bf98882128160-Paper-Datasets_and_Benchmarks.pdf.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 11642–11662. PMLR, 2024. URL <https://proceedings.mlr.press/v235/drouin24a.html>.
- Divyansh Garg, Shaun VanWeelden, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Tomas Abraham, Michael Lara, Federico

- Lopez, James Liu, Atharva Gundawar, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, Christian Schroeder de Witt, and Sumeet Motwani. REAL: Benchmarking autonomous agents on deterministic simulations of real web-sites. *arXiv preprint arXiv:2504.11543*, 2025. doi: 10.48550/arXiv.2504.11543. URL <https://arxiv.org/abs/2504.11543>.
- Google DeepMind. Gemini 3 flash. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>, 2025. Model card.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6864–6890. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.371. URL <https://aclanthology.org/2024.acl-long.371/>.
- Su Kara, Fazle Faisal, and Suman Nath. WABER: Evaluating reliability and efficiency of web agents with existing benchmarks. In *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025a. URL <https://openreview.net/forum?id=HY0VG19fHv>.
- Su Kara, Fazle Faisal, and Suman Nath. WAREX: Web agent reliability evaluation on existing benchmarks. *arXiv preprint arXiv:2510.03285*, 2025b. doi: 10.48550/arXiv.2510.03285. URL <https://arxiv.org/abs/2510.03285>.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 881–905. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.50. URL <https://aclanthology.org/2024.acl-long.50/>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626. Association for Computing Machinery, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Kenunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=5298fKGmv3>. Expert Certification.
- Microsoft. Playwright: Fast and reliable end-to-end testing for modern web apps. <https://playwright.dev>, 2025. Accessed: 2026-05-06.
- Hadi Nekoei, Aman Jaiswal, Patrice Bechard, Oleh Shliazhko, Orlando Marquez Ayala, Mathieu Reymond, Massimo Caccia, Alexandre Drouin, Sarath Chandar, and Alexandre Lacoste. JEF-Hinter: Leveraging offline knowledge for improving web agents adaptation. *arXiv preprint arXiv:2510.04373*, 2025. doi: 10.48550/arXiv.2510.04373. URL <https://arxiv.org/abs/2510.04373>.
- OpenAI. GPT-5.4 Thinking system card. <https://openai.com/index/gpt-5-4-thinking-system-card/>, March 2026a. System card.
- OpenAI. GPT-5.4 mini. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>, 2026b. Model documentation.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Viraj Prabhu, Yutong Dai, Matthew Fernandez, Krithika Ramakrishnan, Jing Gu, Yanqi Luo, Silvio Savarese, Caiming Xiong, Junnan Li, Zeyuan Chen, and Ran Xu. WALT: Web agents that learn tools. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=cgIDqcJcoI>.
- Qwen Team. Qwen3.6-27B. <https://huggingface.co/Qwen/Qwen3.6-27B>, 2026. Model card.

- Yueqi Song, Frank F. Xu, Shuyan Zhou, and Graham Neubig. Beyond browsing: API-based web agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 11066–11085. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.577. URL <https://aclanthology.org/2025.findings-acl.577/>.
- Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. Webxskill: Skill learning for autonomous web agents. *arXiv preprint arXiv:2604.13318*, 2026. doi: 10.48550/arXiv.2604.13318. URL <https://arxiv.org/abs/2604.13318>.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. In *Second Conference on Language Modeling*, 2025a. URL <https://openreview.net/forum?id=lsAY6fWsog>.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Forty-second International Conference on Machine Learning*, 2025b. URL <https://openreview.net/forum?id=NTAhi2JEEE>.
- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? Assessing the current state of web agents. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=6jZi4HSs6o>.
- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. Plugmem: A task-agnostic plugin memory module for LLM agents. In *ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving*, 2026. URL <https://openreview.net/forum?id=LE664kai2e>.
- Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. Polyskill: Learning generalizable skills through polymorphic abstraction for continual learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=KdEsujiSV>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025. doi: 10.48550/arXiv.2504.07079. URL <https://arxiv.org/abs/2504.07079>.
- Shuyan Zhou. WebArena-Infinity: Generating browser environments with verifiable tasks at scale. <https://webarena.dev/webarena-infinity/>, 2026. Accessed 2026-05-24.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oKn9c6ytLx>.

A Implementation Details

Vanilla-IB, AWM, and ASI are all run using the unified codebase and prompts released by Wang et al. (2025a) to ensure that differences in results cannot be attributed to inconsistencies in agent scaffolding, prompt formatting, or environment interaction. The codebase uses BrowserGym (Le Sellier de Chezelles et al., 2025) for interacting with the environment. AWM and ASI come from the same research group (Wang et al., 2025b,a), and AWM is included as a baseline in the more recent ASI codebase. ReasoningBank is run using the codebase released by Ouyang et al. (2026) due to its unique prompt structure and agent logic.

We set the actor LLM temperature of Vanilla-IB to 0, the same as the default value in AWM and ASI. ReasoningBank sets the temperature to 0.7 and we kept the default intact. The auxiliary modules of AWM, ASI, and ReasoningBank use their respective defaults, which we do not modify. ReasoningBank’s codebase also ships a vendored WebArena with custom evaluation pipeline that replaces the standard BrowserGym harness. All methods in this paper are evaluated using the unmodified BrowserGym evaluation harness to ensure identical assessment criteria across configurations.

We host Qwen 3.6-27B on four NVIDIA H100 GPUs using vLLM (Kwon et al., 2023). We used AI assistants during implementation and manuscript preparation for code-related tasks, including generating plotting scripts and LaTeX snippets. We also used them to assist with writing, and for polishing and shortening paragraphs.

B Accessibility Tree Pruning

The accessibility-tree simplification used in Vanilla-IB is a deterministic rule-based function with no LLM call, model fine-tuning, or learned component, and therefore no additional token cost. The function operates on the BrowserGym-style accessibility tree, in which each node has a role (*e.g.*, `button`, `StaticText`, `heading`) and an accessible name, and is rendered as one indented

line per node.

Pruning rule. A `StaticText` child node is removed when its text content is already contained verbatim in the accessible name of its immediate parent. This handles a common pattern in web accessibility trees, where a parent node (such as a labelled button or a list item) carries the same text as its `StaticText` child. Removing the duplicate cuts context length without dropping information. `StaticText` nodes whose stripped content is empty are also removed. Non-`StaticText` nodes, node IDs (BIDs), roles, and tree structure are not modified. The rule has one exception: a `StaticText` child is kept whenever its parent’s role is one of `{article, paragraph, heading, strong, emphasis, mark, sectionheader}`, even when the child’s text is already in the parent’s name. This avoids hollowing out actual prose blocks where the `StaticText` child is the canonical carrier of the rendered text.

Icon characters. Before comparison, Unicode private-use characters in the range U+E600–U+E6FF (typically icon-font glyphs that render as visually empty placeholders) are stripped from both the parent name and the `StaticText` child. Literal `\uXXXX` escapes that appear in the serialized tree are also stripped. This prevents icon characters from blocking an otherwise valid redundancy match.

C Prompt Templates

Vanilla-IB, AWM, and ASI use the prompt templates released by Wang et al. (2025a) without modification, except as noted below. Since the templates are publicly available in the ASI code repository, we do not report them here.

Model-specific modification. The prompts are model-agnostic with one exception. When running experiments with GPT-5.4-mini, we observed two behaviors that interfered with evaluation. First, the model frequently produced action outputs with no accompanying reasoning or explanation. Such explanations are important

for augmentation methods: the induction modules parse them to extract reusable knowledge from trajectories. Second, the model sometimes generated turns asking the user for clarification or additional input, which is not possible during automated benchmark evaluation. To address both issues, the following text was appended to the action prompt for GPT-5.4-mini:

You cannot ask for follow-up questions or hand back to the user. Do your best with the information already available. Output a summary of your reasoning and the important information you have found, followed by the best actions at this step (in one triple backticks block).

It is worth noting that Vanilla-IB is less dependent on reasoning traces in the output: the vanilla agent acts only on the current observation, so in many cases explanatory text is overhead that could in principle be removed to reduce completion tokens. This represents an additional efficiency advantage of Vanilla-IB that our numbers do not capture. However, to ensure a fair comparison across all methods, we applied the same prompt to all methods under each model, including the above addition for GPT-5.4-mini.

D Failure Mode Analysis of Augmented Agents

Section 5.2 discusses the structural fragility of programmatic skills in dynamic accessibility-tree interfaces at a conceptual level. This appendix extends that discussion with quantitative characterizations of failure modes across all three augmented methods, and presents concrete examples of tasks where augmented agents fail while Vanilla-IB succeeds. Unless stated otherwise, quantities are averaged over three independent runs per domain-model configuration.

D.1 ASI: Verification Confounds Function Success with Agent Recovery

ASI induces Python functions from successful trajectories and adds them to a shared action library after a verification episode. The intended

guarantee is that only working functions enter the library. In the verification procedure, the induced function is called as the first decision. If the function changes the environment but fails on one of its underlying actions, the agent may still recover via primitive actions and solve the task. In that case, the episode can be judged a success and the function is written to disk, without actual verification. A common instance of this failure is when the function fails because an element ID has changed after the page reloaded for the verification episode. Functions that require arguments whose values depend on a prior action’s page state are especially prone to this: the caller cannot supply a valid BID for an element that only becomes visible after an earlier click. The induced functions are also typically short, averaging approximately 2.4 primitive actions per function under Gemini 3 Flash and 2.5 under GPT-5.4-mini, which limits the abstraction benefit. Some functions also hardcode task-specific values that do not generalize across tasks; for example, the function `fill_refund_request_form` in the shopping library hardcodes the complaint message “*It broke after just three days of use*” regardless of the actual reason for the refund.

Table 6 quantifies the verification-confound effect. We define a *first-step verification failure* as a verification run in which the induced function fails before the second agent action. A *recovered* verification is one that was still judged correct despite that first-step failure. First-step failures are not rare. Under GPT, the induced function fails immediately in 72.2% of Shopping verifications; over half of those are recovered and still produce a stored library entry. Even under Gemini Shopping, where the first-step failure rate is lower (33.3%), two thirds of failures recover. In both cases the result is a function library that contains entries whose correctness at verification time was not due to the function itself.

Domain	Attempts	First-step fail	Recov. rate
Gemini 3 Flash			
Shopping	58.0	19.3	69.0%
Reddit	30.0	9.0	59.3%
Admin	64.3	6.3	63.2%
GPT-5.4-mini			
Shopping	44.3	32.0	54.2%
Reddit	25.0	12.3	45.9%
Admin	51.7	22.7	47.1%

Table 6 ASI verification failures at the first high-level function call on WebArena. Values are averages over three replicas. *Attempts*: total number of function induction attempts by ASI across all tasks. *First-step fail*: number of attempts where the induced function failed before the second agent action. *Recov. rate*: fraction of first-step failures where the verification episode was nonetheless judged correct, causing a potentially broken function to be stored in the shared library.

D.2 AWM: Generic Templates and Induction from Failed Trajectories

AWM induces abstract natural-language workflows from trajectories the judge deems successful, then retrieves and injects the closest workflow into the actor’s prompt at each inference step. Many induced workflows are too generic to add navigational value. A stored shopping workflow for order-history aggregation reduces to: “*click the account link, click the full-order-history link, send a summary*”. This describes the high-level intent without preserving operational details that determine correctness, such as whether pagination is needed, what the stopping condition is, or how totals are computed. Similarly, a stored price-range workflow directs the agent to sort, toggle direction, navigate to the last page, and report the range, but omits checks for sort direction availability, filtered result sets, or whether the last-page element is reachable in the current state. Because the workflow is injected into the actor’s prompt, the agent must decide at each step whether to follow it or ignore it, a burden that smaller models are especially ill-equipped to handle, as it competes with the current accessibility tree and task instruction.

A more fundamental problem is that AWM’s

induction is gated only by the judge’s verdict. Table 7 shows the fraction of workflow-induction events triggered by failed trajectories. On Shopping, nearly half of all induction events, 49.5% under Gemini and 52.3% under GPT, originate from failed agent trajectories. The contamination is substantially lower for Reddit (11–19%), which may help explain AWM’s stronger performance on that domain. Because AWM’s duplicate-suppression mechanism checks only whether a new workflow semantically overlaps an existing one, it does not filter out strategies that are simply wrong: once an incorrect workflow enters the library, it can be retrieved and applied to all future tasks in the same category.

Domain	Induced	From failed (%)	Final WFs
Gemini 3 Flash			
Shopping	105.7	49.5%	37.3
Reddit	45.7	10.9%	16.7
Admin	99.7	42.1%	48.3
GPT-5.4-mini			
Shopping	78.3	52.3%	33.7
Reddit	31.3	19.1%	13.7
Admin	50.0	42.0%	24.3

Table 7 AWM workflow-induction statistics on WebArena, averaged per run. *Induced*: total induction events triggered by the judge for a run. *From failed (%)*: fraction originating from tasks the ground-truth evaluator classified as failed. *Final WFs*: distinct workflows in the final library after duplicate suppression.

D.3 ReasoningBank: Contamination and Memory Over-Application

ReasoningBank stores a few short lessons from every task trajectory regardless of outcome, labeling each entry as a success or failure based on the judge’s verdict. The K most semantically similar entries are retrieved at inference time and injected into the actor’s context. Table 8 reports the size and contamination of the resulting memory corpus. On Shopping tasks, more than half of all success-labeled entries, 52.9% for Gemini and 59.5% for GPT, come from trajectories that failed according to the ground-truth evaluator. These entries are framed with positive guidance

language regardless of their true provenance, so the retrieval mechanism cannot distinguish between lessons that worked and lessons that did not.

Domain	Total mem.	Succ.-labeled	FP in succ. (%)	FN in fail (%)
Gemini 3 Flash				
Shopping	518	96	52.9%	40.7%
Reddit	294	32	30.2%	25.3%
Admin	507	86	42.6%	40.4%
GPT-5.4-mini				
Shopping	544	67	59.5%	18.4%
Reddit	307	20	60.0%	17.4%
Admin	532	62	48.1%	25.2%

Table 8 ReasoningBank memory-corpus statistics on WebArena, averaged over three runs. Tasks processed per run: 187 (Shopping), 106 (Reddit), 182 (Admin). *Succ.-labeled*: tasks labeled successful by the judge. *FP in succ.*: fraction of success-labeled tasks from trajectories that failed according to the ground-truth evaluator. *FN in fail*: fraction of fail-labeled tasks from trajectories that succeeded according to the ground-truth evaluator.

A separate concern is injection pressure. Even though only a few memories are retrieved per task, they are referenced heavily in the agent traces: across all domain-model configurations, we observe an average of 8–12 explicit memory-item mentions per task, with over 95% of Gemini tasks and over 75% of GPT tasks referencing at least one retrieved memory. The retrieved memories therefore function as active policy hints rather than passive background. When a memory is overly general or mildly mismatched to the current task, the agent may prioritize it over the current accessibility tree, producing extra navigation steps, unnecessary checks, or incorrect early termination. Vanilla-IB, which carries no retrieved memory layer, avoids this class of distraction entirely by grounding each action in the current observation.

D.4 Discussion: Structural Limits of Judge-Based Quality Control

The contamination patterns described above for AWM and ReasoningBank, and the verification

confound in ASI, share a common structural root: each method delegates quality control to an LLM judge whose input is fundamentally incomplete for the tasks it is asked to evaluate.

The judge receives the agent’s textual output (its reasoning trace and final response) and, in some configurations, a snapshot of the browser state at the end of the episode. It does not observe the sequence of intermediate page states the agent traversed, the specific elements it interacted with, or whether information the agent reports was actually encountered during navigation or confabulated in the reasoning trace. For tasks where correctness depends on having visited the right pages and extracted the right values, a judge working from this terminal view cannot reliably reconstruct whether the work was actually done.

This is not necessarily a matter of judge capability. A more powerful model receiving the same partial input faces the same gap: the evidence needed to verify navigation-dependent correctness is simply absent from the judge’s context. Nor is it straightforwardly fixable by expanding what is passed to the judge. Supplying the full trace at every step would require passing on the order of hundreds of thousands of tokens per judge call for a ten-step task on a content-heavy page, making each evaluation call more expensive than the agent trajectory it is judging, and possibly exceeding practical context limits entirely.

D.5 Illustrative Task-Level Failures

Table 9 presents representative task-level failures drawn from our Gemini 3 Flash runs, showing in each case what the augmented agent does and how Vanilla-IB succeeds on the same task.

E Per-Run Variance Results

Tables 10, 11, and 12 report success rate and token usage for each method across three independent runs under Gemini 3 Flash, GPT-5.4-mini, and Qwen 3.6-27B, respectively. Standard deviations are sample std over the three runs.

Table 9 Example failures of augmented agents where Vanilla-IB succeeds. Tasks are selected from WebArena (single runs shown; all from Gemini 3 Flash).

Method	Intent	Augmented agent behavior	Vanilla-IB behavior
AWM	“Buy the highest-rated product in Ceiling Light within a budget above \$1000.”	Retrieved workflow for “Buy the highest-rated product” prescribes sorting then clicking the top result. Agent buys the wrong product because the workflow does not account for rating ties or the budget threshold.	Applies sorting, checks rating and price against the budget, and selects the correct product.
ASI	“Change the delivery address for my most recent order to 3 Oxford St, Cambridge, MA.”	Calls <code>navigate_to_contact_page</code> and <code>fill_contact_form</code> using the element ID from the docstring example ('1381'). Receives <code>ValueError</code> and <code>TypeError</code> ; form submission fails.	Navigates to Address Book, adds the new address, recognizes that the order address cannot be changed through the UI, and reports accordingly.
ReasoningBank	“What are the top-5 best-selling products in 2023?”	Retrieved memory on handling ranking ties directs attention to tie-breaking. Agent misidentifies the 4th and 5th items (reports <i>Sparta Gym Tank</i> and <i>Angel Light Running Short</i> instead of <i>Sprite Stasis Ball</i> and <i>Hawkeye Yoga Short</i>).	Reads the same report without distraction and returns the correct ranked list.

Method	Success Rate (%) $\uparrow$				Tok/task (K) $\downarrow$			
	Run a	Run b	Run c	Mean $\pm$ Std	Run a	Run b	Run c	Mean $\pm$ Std
Shopping								
Vanilla-IB	47.59	49.73	45.99	47.77 $\pm$ 1.9	46.6	45.5	45.1	45.7 $\pm$ 0.8
AWM	43.32	39.04	41.18	41.18 $\pm$ 2.1	68.0	70.6	73.9	70.8 $\pm$ 3.0
ASI	43.32	45.45	45.45	44.74 $\pm$ 1.2	82.4	83.0	83.0	82.8 $\pm$ 0.3
ReasoningBank	44.39	44.91	47.05	45.45 $\pm$ 1.4	56.0	54.1	53.6	54.6 $\pm$ 1.3
Reddit								
Vanilla-IB	48.11	47.17	47.17	47.48 $\pm$ 0.5	71.7	69.5	73.8	71.7 $\pm$ 2.2
AWM	49.06	46.23	47.17	47.48 $\pm$ 1.4	87.6	88.9	85.0	87.2 $\pm$ 2.0
ASI	47.17	43.40	44.34	44.97 $\pm$ 2.0	98.5	89.3	96.0	94.6 $\pm$ 4.8
ReasoningBank	39.62	38.68	41.50	39.93 $\pm$ 1.4	78.9	73.6	77.8	76.8 $\pm$ 2.8
Admin								
Vanilla-IB	53.85	56.59	56.59	55.68 $\pm$ 1.6	95.2	98.6	103.2	99.0 $\pm$ 4.0
AWM	47.80	47.80	46.70	47.43 $\pm$ 0.6	136.6	147.0	144.4	142.7 $\pm$ 5.4
ASI	53.85	51.65	52.75	52.75 $\pm$ 1.1	132.0	152.3	133.9	139.4 $\pm$ 11.2
ReasoningBank	47.80	51.10	47.80	48.90 $\pm$ 1.9	120.8	126.4	126.5	124.6 $\pm$ 3.3

Table 10 Per-run success rate (%) and token usage (K per task) across three independent runs under Gemini 3 Flash on WebArena. Standard deviations are sample std over the three runs.

Method	Success Rate (%) $\uparrow$				Tok/task (K) $\downarrow$			
	Run a	Run b	Run c	Mean $\pm$ Std	Run a	Run b	Run c	Mean $\pm$ Std
Shopping								
Vanilla-IB	37.43	39.04	39.04	38.50 $\pm$ 0.9	55.1	53.5	54.2	54.3 $\pm$ 0.8
AWM	27.81	31.02	31.02	29.95 $\pm$ 1.9	64.2	61.6	60.2	62.0 $\pm$ 2.0
ASI	34.22	32.62	32.62	33.15 $\pm$ 0.9	70.6	83.5	66.2	73.4 $\pm$ 9.0
ReasoningBank	21.93	27.81	29.41	26.38 $\pm$ 3.9	59.3	58.2	56.2	57.9 $\pm$ 1.6
Reddit								
Vanilla-IB	38.68	35.85	29.25	34.59 $\pm$ 4.8	89.4	102.5	96.6	96.2 $\pm$ 6.6
AWM	30.19	30.19	28.30	29.56 $\pm$ 1.1	77.1	94.4	69.8	80.4 $\pm$ 12.6
ASI	26.42	30.19	30.19	28.93 $\pm$ 2.2	83.8	85.0	92.8	87.2 $\pm$ 4.9
ReasoningBank	18.87	24.53	22.64	22.01 $\pm$ 2.9	67.8	72.5	67.3	69.2 $\pm$ 2.9
Admin								
Vanilla-IB	35.16	35.16	37.36	35.89 $\pm$ 1.3	117.4	133.0	121.9	124.1 $\pm$ 8.0
AWM	28.57	33.52	34.62	32.24 $\pm$ 3.2	129.3	95.9	114.9	113.4 $\pm$ 16.8
ASI	28.57	35.16	35.16	32.96 $\pm$ 3.8	126.1	146.5	142.9	138.5 $\pm$ 10.9
ReasoningBank	36.26	32.97	33.52	34.25 $\pm$ 1.8	121.0	125.5	122.8	123.1 $\pm$ 2.3

Table 11 Per-run success rate (%) and token usage per task (K) under GPT-5.4-mini on WebArena. Standard deviations are sample std over three runs.

Method	Success Rate (%) $\uparrow$				Tok/task (K) $\downarrow$			
	Run a	Run b	Run c	Mean $\pm$ Std	Run a	Run b	Run c	Mean $\pm$ Std
Shopping								
Vanilla-IB	47.06	44.39	46.52	45.99 $\pm$ 1.4	57.9	60.5	57.9	58.8 $\pm$ 1.5
AWM	39.57	43.30	41.18	41.35 $\pm$ 1.9	73.6	66.7	75.1	71.8 $\pm$ 4.5
ASI	47.06	43.85	40.10	43.67 $\pm$ 3.5	72.6	81.1	82.8	78.8 $\pm$ 5.5
ReasoningBank	39.57	40.64	39.04	39.75 $\pm$ 0.8	65.6	65.2	65.3	65.4 $\pm$ 0.2
Reddit								
Vanilla-IB	47.17	43.40	42.45	44.34 $\pm$ 2.5	94.5	86.5	88.0	89.7 $\pm$ 4.3
AWM	44.34	45.28	39.62	43.08 $\pm$ 3.0	110.3	87.8	112.8	103.6 $\pm$ 13.8
ASI	43.40	41.51	44.34	43.08 $\pm$ 1.4	118.1	125.9	117.6	120.5 $\pm$ 4.7
ReasoningBank	39.62	41.51	42.45	41.19 $\pm$ 1.4	96.1	95.9	105.4	99.1 $\pm$ 5.4
Admin								
Vanilla-IB	49.45	52.20	50.54	50.73 $\pm$ 1.4	141.2	125.3	124.4	130.3 $\pm$ 9.5
AWM	45.60	43.40	49.45	46.15 $\pm$ 3.1	192.8	172.3	170.3	178.5 $\pm$ 12.5
ASI	51.65	48.35	47.25	49.08 $\pm$ 2.3	177.7	183.1	185.9	182.2 $\pm$ 4.2
ReasoningBank	46.15	47.25	49.45	47.62 $\pm$ 1.7	160.8	162.9	147.1	156.9 $\pm$ 8.6

Table 12 Per-run success rate (%) and token usage per task (K) under Qwen 3.6-27B on WebArena. Standard deviations are sample std over three runs.